

AhamX Bodhi

Where I Becomes Infinite

Model Sizing and Scaling Laws

LLM Fundamentals · Module 1.2 · Prof. Sashikumaar Ganesan



What We Cover Today

Foundation Needed

- What a language model predicts
- Concept of parameters (weights)
- Basic neural network intuition
- Familiarity with GPT / Llama names

Session Roadmap

- Why model size matters
- How parameters scale with layers
- Scaling Laws: compute, data, size
- Chinchilla vs. GPT-3 trade-offs
- Practical sizing decisions



Learning Objectives

By the End of This Session You Will Be Able To

1. **Explain** what model parameters are and how they are counted
2. **Describe** the relationship between model size and performance
3. **Interpret** Scaling Laws (Kaplan et al. 2020; Hoffmann et al. 2022)
4. **Compare** efficient vs. over-parameterised training regimes
5. **Apply** sizing intuition to choose an appropriate model for a task

What Are Model Parameters?



Definition

A **parameter** is a learnable numerical value (weight or bias) stored inside the model. During training, parameters are adjusted to minimise the prediction loss. At inference, they are frozen.

Where Parameters Live in a Transformer

- **Embedding matrices** — convert tokens to dense vectors
- **Attention projections** — W_Q, W_K, W_V, W_O per head per layer
- **Feed-forward layers** — two large linear projections per layer
- **Layer-norm scales & biases** — small but present in every sub-layer

Parameter Count --- Where the Numbers Come From



Embedding Matrix: $V \times d_{\text{model}}$ (e.g. $50,000 \times 768 \approx 38\text{M}$)

Attention Projections per Layer: $4 \times d_{\text{model}}^2$ (e.g. $4 \times 768^2 \approx 2.4\text{M}$)

Feed-Forward Network per Layer: $8 \times d_{\text{model}}^2$ (e.g. $8 \times 768^2 \approx 4.7\text{M}$)

Layer Norms (all layers): $\approx 2 \times 2 \times d_{\text{model}} \times L$ (small, $< 1\text{M}$)

Total $\approx Vd + L(12d^2) \Rightarrow$ **Scales as** $O(d^2 \cdot L)$

V = vocab size, d = model dimension (hidden size), L = number of layers



Common LLM Sizes at a Glance

Model	Params	Layers	d_{model}	Typical Use
TinyLlama-1.1B	1.1B	22	2048	Edge, mobile, experiments
Llama-3-8B	8B	32	4096	Research, fine-tuning
Llama-3-70B	70B	80	8192	Production chat, reasoning
GPT-3	175B	96	12288	Broad NLP, generation
GPT-4 (est.)	$\sim 1\text{T}$	—	—	Frontier reasoning

Key Observation

- Parameters grow roughly as $d^2 \times L$; doubling d quadruples cost
- Larger models generalise better *if* trained on enough data



Scaling Laws --- Kaplan et al. (2020)

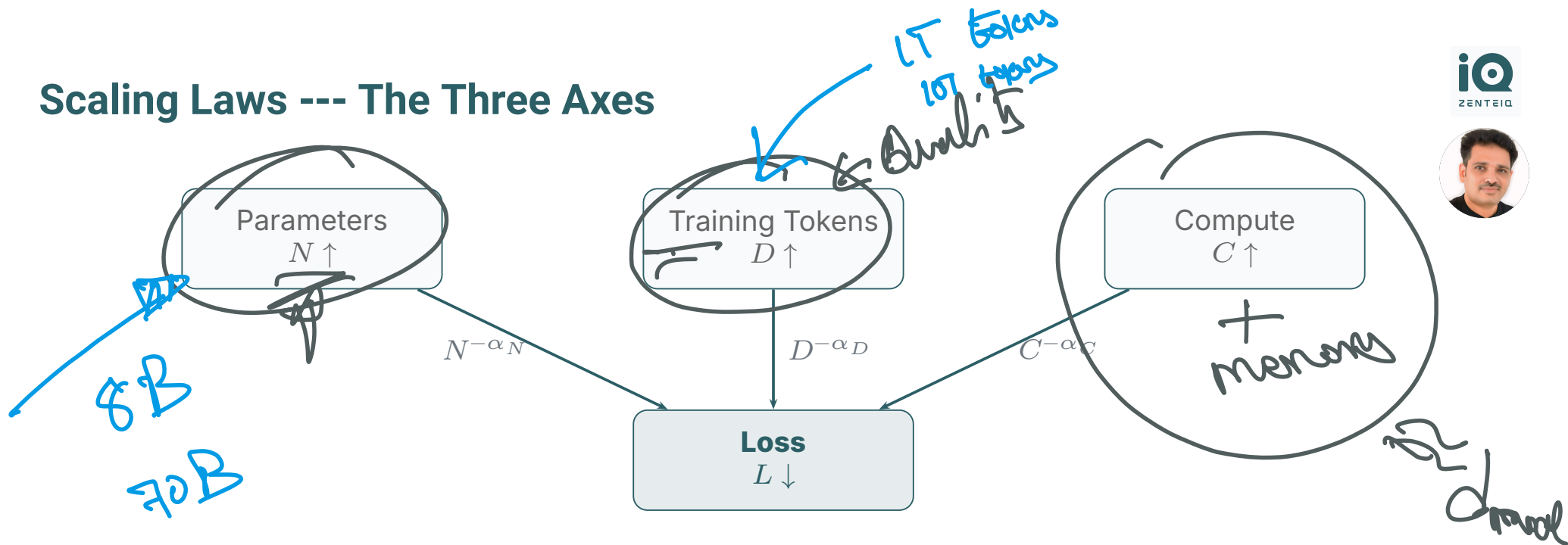
The Key Finding

OpenAI showed that language model **loss** follows smooth **power laws** with respect to three independent axes: number of **parameters** (N), training **tokens** (D), and **compute** budget (C).

Power-Law Relationships (simplified)

- $L(N) \propto N^{-\alpha_N}$ — more parameters $\Rightarrow$ lower loss
- $L(D) \propto D^{-\alpha_D}$ — more data $\Rightarrow$ lower loss
- $L(C) \propto C^{-\alpha_C}$ — more compute $\Rightarrow$ lower loss
- Exponents are consistent across architectures and tasks

Scaling Laws --- The Three Axes



Insight: With fixed compute, the Kaplan laws suggested spending most FLOPS on a *very large* model trained on *fewer* tokens than optimal.

[

Handwritten notes illustrating matrix operations and variables:

$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ $8K \times 1$

$x_1, x_2, \dots$

$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ $8K \times 1$

$x_1, x_2, \dots$

$$\vec{x}_1 = \begin{bmatrix} \vdots \\ \vdots \end{bmatrix} \text{ dimension } n$$

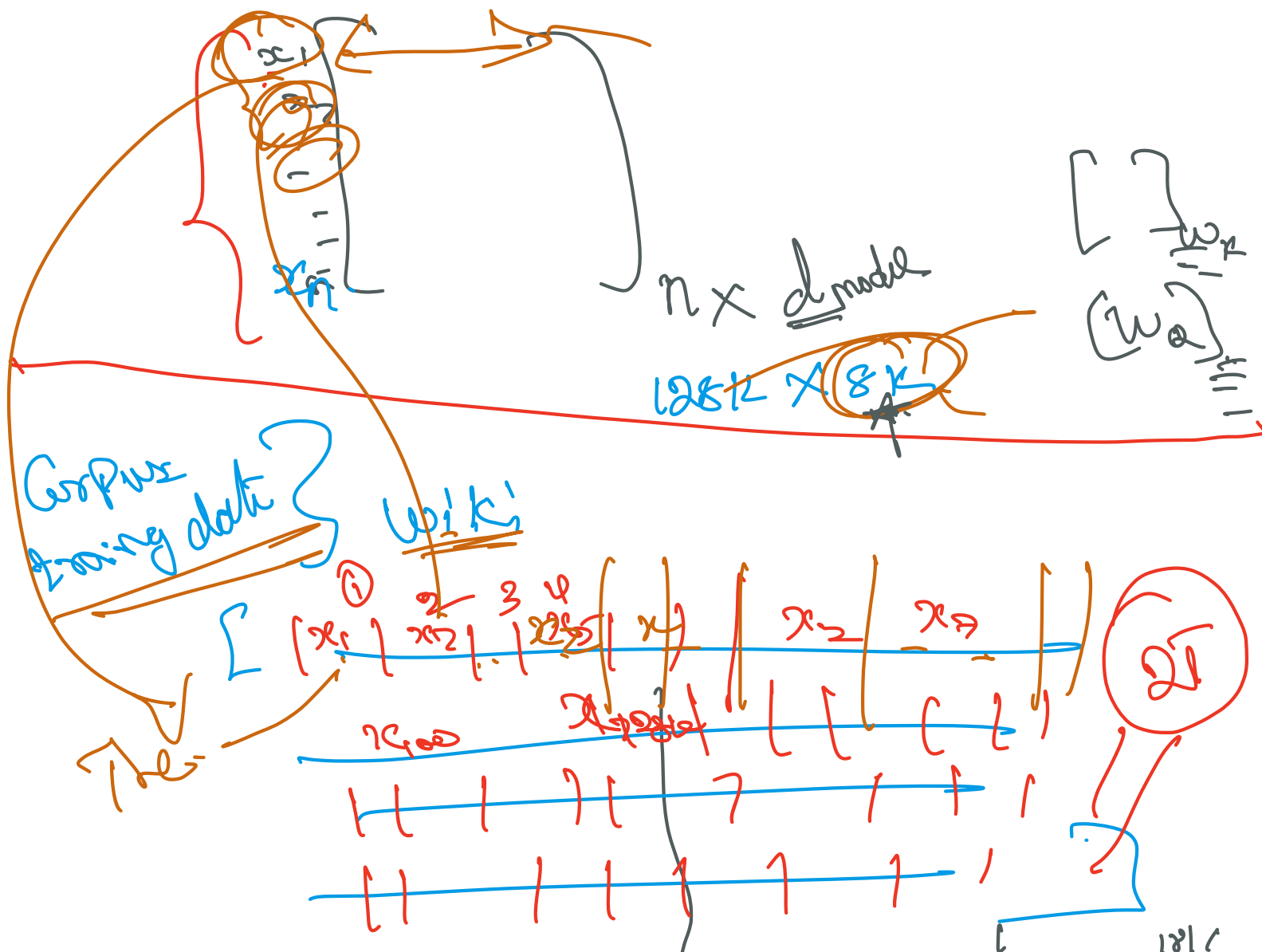
$\vdots$

$$d_{\text{model}} \approx 2^{14} \sim \underline{\underline{8K}}$$

x_m

n — total # tokens in vocab

$$n \approx 55K \rightarrow \underline{\underline{128K}}$$





Chinchilla --- Correcting the Scaling Recipe

DeepMind, 2022: "Training Compute-Optimal LLMs"

Hoffmann et al. trained 400 models across a wide range of sizes and token budgets and found the Kaplan recipe was **wrong**: models were being trained with *too many parameters and too few tokens*.

Chinchilla Optimal Rule

- For a fixed compute budget C , allocate **equally** between N and D
- Optimal: $N_{\text{opt}} \propto C^{0.5}$, $D_{\text{opt}} \propto C^{0.5}$
- Empirical rule: train for $\approx$ **20 tokens per parameter**
- Chinchilla (70B, 1.4T tokens) outperformed GPT-3 (175B, 300B tokens)



Kaplan vs. Chinchilla --- A Comparison

Kaplan et al. 2020

- OpenAI scaling study
- Fix compute $\Rightarrow$ scale N aggressively
- Train on fewer tokens than optimal
- Led to GPT-3 (175B, 300B tokens)
- **Result:** under-trained large models

Hoffmann et al. 2022

- DeepMind Chinchilla study
- Fix compute $\Rightarrow$ balance N & D
- ~ 20 tokens per parameter
- Led to Chinchilla (70B, 1.4T tokens)
- **Result:** smaller but compute-optimal



Practical Impact of Chinchilla

Industry Shift

Chinchilla triggered a paradigm change: the field moved from chasing *parameter records* to chasing *token-efficient training*.

What Changed in Practice

- **Llama family** — Meta trained Llama-2-7B on 2T tokens (>280 tokens/param)
- **Mistral 7B** — beats Llama-2-13B despite half the parameters
- **Phi models** — Microsoft showed high-quality data beats raw scale
- **Inference cost** — smaller, well-trained models are cheaper to serve



Compute Budget and FLOP Estimates

Counting Training FLOPs

Training a dense transformer for one step on one token costs approximately $6N$ floating-point operations, where N is the parameter count. Total training FLOPs: $C \approx 6ND$.

Quick Estimates

- GPT-3 (175B, 300B tokens): $C \approx 3.1 \times 10^{23}$ FLOPs
- Chinchilla (70B, 1.4T tokens): $C \approx 5.8 \times 10^{23}$ FLOPs
- Llama-3-8B (8B, 15T tokens): $C \approx 7.2 \times 10^{23}$ FLOPs
- 10^{23} FLOPs $\approx$ one A100 GPU running for ~ 3 years

2 A100

1.5 yrs
 ≈ 2 yrs

NO



GenAI Application --- Choosing the Right Model Size

The Practitioner's Question

You have a budget and a task. Should you use a 7B, 13B, or 70B model? Scaling laws give you a *principled* answer rather than guesswork.

Decision Framework

- **Edge / real-time** (<100 ms latency): TinyLlama-1.1B, Phi-2 (2.7B)
- **Research / fine-tuning**: Llama-3-8B — well-trained, open weights
- **Production chat & reasoning**: Llama-3-70B, Mixtral-8×7B (MoE)
- **Frontier tasks** (coding, science): GPT-4, Claude 3 Opus, Gemini Ultra
- **Rule of thumb**: prefer a smaller model trained on more data over a large model trained on fewer tokens



GenAI Application --- Scaling in Fine-Tuning

Fine-Tuning Scaling Insight

Scaling laws also apply to fine-tuning: *more fine-tuning data* can compensate for a smaller base model. This underpins PEFT methods like LoRA and QLoRA that efficiently adapt mid-sized models.

Real-World Fine-Tuning Decisions

- Healthcare NLP: Llama-3-8B fine-tuned on 100K clinical notes often outperforms GPT-3 on domain-specific tasks
- Legal summarisation: 7B + LoRA matches 70B zero-shot on court documents
- Code generation: DeepSeek-Coder-6.7B fine-tuned beats GPT-3.5 on HumanEval
- Key takeaway: *domain data quality > raw parameter count*



GenAI Application --- Emergent Abilities at Scale

What "Emergence" Means

Certain capabilities (multi-step arithmetic, chain-of-thought reasoning, in-context learning) appear *discontinuously* above a threshold model size and cannot be predicted by smooth power-law extrapolation.

Emergence Thresholds (approximate)

- **In-context few-shot learning:** emerges reliably above $\sim 7\text{B}$ params
- **Multi-step math:** reliably above $\sim 50\text{B}$ params (or CoT prompting)
- **Code generation:** usable quality above $\sim 6\text{B}$ code-specialised params
- **Instruction following:** requires RLHF/SFT alignment, not just scale
- **Implication:** target task determines minimum viable model size



GenAI Application --- Mixture of Experts (MoE)

Beyond Dense Scaling

Mixture-of-Experts architectures decouple *total parameter count* from *active parameters per token*, offering a new axis along which to scale.

MoE Scaling Examples

- **Mixtral-8×7B**: 46.7B total params, 12.9B active per token
- **GPT-4**: rumoured to use ~8 experts of ~110B each
- MoE models have lower *inference* FLOPs than their parameter count implies
- Trade-off: higher *memory* footprint; expert routing adds complexity
- Scaling law research for MoE is an active open problem

Skills Developed



Conceptual Skills

- Interpreting power-law plots
- Understanding parameter vs. FLOP trade-offs
- Reading model cards and benchmarks
- Recognising compute-optimal regimes

Applied Skills

- Estimating parameter counts from architecture specs
- Selecting model size for a task & budget
- Evaluating published scaling claims
- Advising on fine-tuning vs. pre-training cost



Professional Roles and Relevance

Role	How Scaling Laws Apply
ML Engineer	Choosing model size & training budget for production systems
AI Research Scientist	Designing experiments guided by scaling predictions
Data Engineer	Estimating data requirements for a target model quality
AI Product Manager	Aligning model capability roadmaps with compute costs
Software Developer	Understanding API latency vs. model size trade-offs



Knowledge Graph --- Prerequisites

Concepts Required Before This Session

- **AI-LM-FN-TH-000001** — What is a Language Model?
- **AI-LM-FN-TH-000002** — Probability and Token Prediction
- **AI-LM-FN-TH-000003** — Autoregressive Generation
- **AI-GN-FN-TH-000001** — What is Generative AI? (Module 1.1)



What This Concept Unlocks --- Part 1

Downstream Concepts Enabled

- **AI-LM-FN-TH-000005** — Foundation Models Overview
- **AI-LM-FN-AI-000001** — Foundation Model Comparison (GPT, Llama, TinyLlama)
- **AI-LM-IN-TH-000001** — API Fundamentals for LLMs (Module 3)
- **AI-PT-IN-TH-000001** — Pre-Training Overview (Module 9)



What This Concept Unlocks --- Part 2

Advanced Concepts Enabled

- **AI-PT-AD-TH-000001** — Chinchilla Scaling Laws (Module 9, deep dive)
- **AI-PT-AD-TH-000002** — Compute-Optimal Training (Module 9)
- **AI-FT-IN-TH-000001** — What is Fine-Tuning? (Module 7)
- **AI-OP-IN-TH-000002** — Quantization and Compression (Module 9)



Related Concepts in the Curriculum

Cross-Module Connections

- **AI-TR-FN-TH-000001/000005** — Transformer architecture that hosts the parameters
- **AI-FT-IN-TH-000006/000007** — LoRA / QLoRA exploit scaling insight to reduce cost
- **AI-RL-IN-TH-000001** — RLHF alignment often applied *after* scaling
- **AI-OP-IN-TH-000001** — Infrastructure requirements driven by model size
- **AI-DF-IN-TH-000007/000008** — Data / model parallelism needed for large-scale training



Common Misconceptions

Watch Out For These

- **"Bigger is always better"** — Chinchilla shows data efficiency matters as much
- **"Parameter count = model quality"** — Training data, quality, and alignment matter equally
- **"Scaling laws predict emergent abilities"** — Emergence is *not* captured by smooth power laws
- **"FLOPs = tokens × params"** — Correct formula is $C \approx 6ND$; the factor of 6 is often forgotten

Summary



Key Takeaways from This Session

1. **Parameters** are learnable weights; count scales as $O(d^2 \cdot L)$
2. **Kaplan (2020)** identified power-law loss scaling with N, D, C
3. **Chinchilla (2022)** corrected the recipe: balance parameters and tokens equally
4. **Rule of thumb**: ~ 20 tokens per parameter for compute-optimal training
5. **Practical**: prefer smaller, data-rich models over large, data-starved ones

Next Steps



Immediately After This Session

- Browse model cards on Hugging Face Hub
- Note parameter counts & training tokens
- Study the Chinchilla paper abstract
- Try the LLM Visualizer tool online

Upcoming in Module 1.2

- Foundation Models Overview (next concept)
- Foundation Model Comparison Lab
- Module 1 Assessment & Mini-Project

Thank You

Model Sizing and Scaling Laws · AI-LM-FN-TH-000004